



Chapter 17

Interpolation

By

Hyung Jae (Chris) Chang

Troy University



Introduction

- A dataset consisting of independent data values, x_i and dependent data values, y_i where $i = 1, 2, \dots, n$
- Goal: find an estimation function $\hat{y}$ such that $\hat{y}(x_i) = y_i$ for every point in our dataset.
- Given a new x^* , we can interpolate its function value using $\hat{y}(x^*)$ which is called interpolation function.

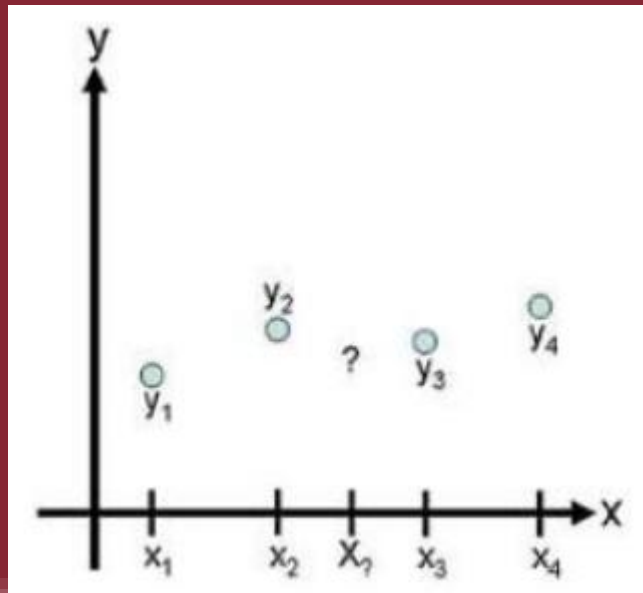


Illustration of the interpolation problem: estimate the value of a function in-between data points.



Linear Interpolation

- In linear interpolation, the estimated point is assumed to lie on the line joining the nearest points to the left and right.
- Given x-data points in ascending order ($x_i < x_{i+1}$) and x being a point such that $x_i < x < x_{i+1}$, the linear interpolation at x is

$$\hat{y}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{(x_{i+1} - x_i)}$$



Example

TRY IT! Find the linear interpolation at $x = 1.5$ based on the data $x = [0, 1, 2]$, $y = [1, 3, 2]$. Verify the result using SciPy's function `interp1d`.

Since $1 < x < 2$, we use the second and third data points to compute the linear interpolation. Plugging in the corresponding values gives

$$\hat{y}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{(x_{i+1} - x_i)} = 3 + \frac{(2 - 3)(1.5 - 1)}{(2 - 1)} = 2.5.$$

```
In [1]: from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

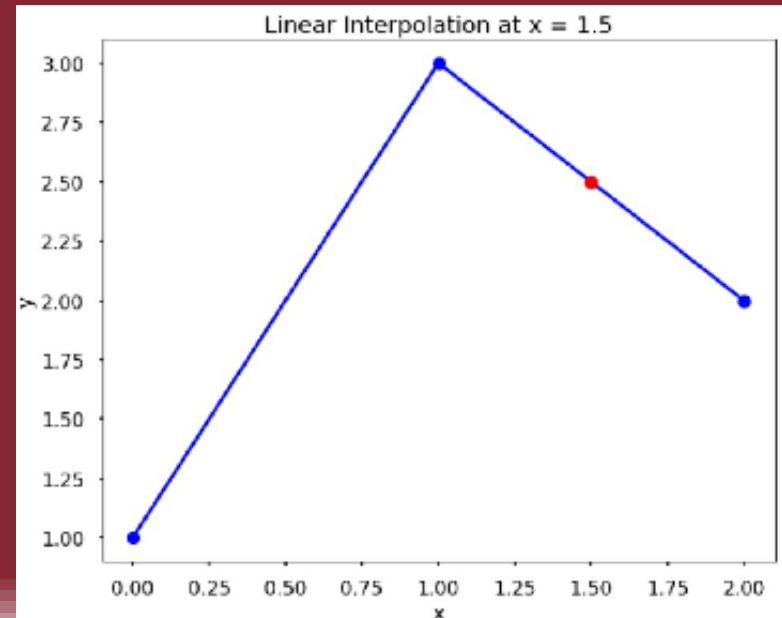
plt.style.use("seaborn-poster")

In [2]: x = [0, 1, 2]
y = [1, 3, 2]

f = interp1d(x, y)
y_hat = f(1.5)
print(y_hat)

2.5

In [3]: plt.figure(figsize = (10,8))
plt.plot(x, y, "-ob")
plt.plot(1.5, y_hat, "ro")
plt.title("Linear Interpolation at x = 1.5")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```





Cubic Spline Interpolation

- In cubic spline interpolation, the interpolation function is a set of piecewise cubic functions with an assumption that the points (x_i, y_i) and (x_{i+1}, y_{i+1}) are joined by a cubic polynomial $S_i(x)$.

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

which is valid for $x_i \leq x \leq x_{i+1}$ for $i = 1, 2, \dots, n-1$

- To find the interpolation function, we need to first find coefficients a_i, b_i, c_i, d_i for each of the cubic functions
- For n points, there are $n-1$ cubic functions to find total $4(n-1)$ coefficient
- Need $4(n-1)$ independent equations to find all the coefficients!



Cubic Spline Interpolation

- $S_i(x_i) = y_i$
- $S_i(x_{i+1}) = y_{i+1}$
 - Cubic functions must intersect data points on the left and the right
 - Which gives us $2(n-1)$ equations
- $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$
- $S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$
 - Cubic function to join as smoothly with its neighbors as possible
→ constrain the splines to have continuous first and second derivatives at data points $i = 2, \dots, n-1$
 - Which gives us $2(n-2)$ equations
- $S''_1(x_1) = 0$
- $S''_{n-1}(x_n) = 0$
 - Arbitrary. Chosen to fit the circumstances of the interpolation.
 - A common set of final constraints is to assume that the second derivatives are 0 at endpoints → the curve is a straight line at end points.



Cubic Spline Interpolation in Python

- Use SciPy's function, CubicSpline.

TRY IT! Use CubicSpline to plot the cubic spline interpolation of the dataset $x = [0, 1, 2]$ and $y = [1, 3, 2]$ for $0 \leq x \leq 2$.

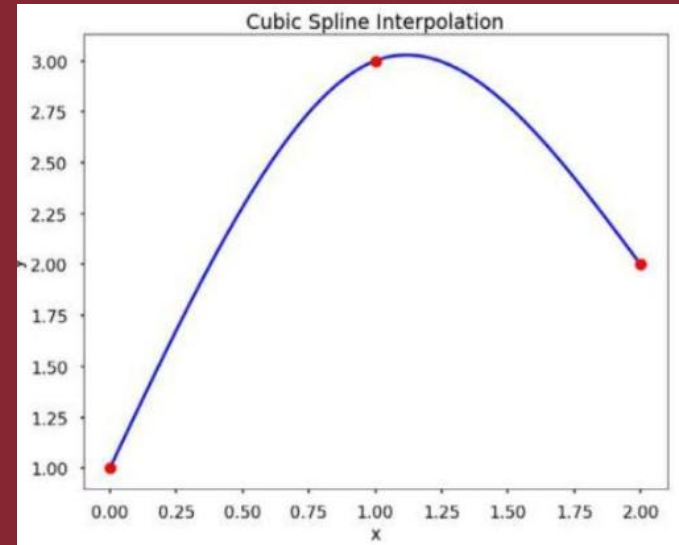
```
In [1]: from scipy.interpolate import CubicSpline
import numpy as np
import matplotlib.pyplot as plt

plt.style.use("seaborn-poster")

In [2]: x = [0, 1, 2]
y = [1, 3, 2]

# use bc_type = "natural" adds the constraints
f = CubicSpline(x, y, bc_type="natural")
x_new = np.linspace(0, 2, 100)
y_new = f(x_new)

In [3]: plt.figure(figsize = (10,8))
plt.plot(x_new, y_new, "b")
plt.plot(x, y, "ro")
plt.title("Cubic Spline Interpolation")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```





Finding Coefficients

- Uses system of linear equations with $4(n-1)$ unknowns for the constraints.
- For n data points, the unknowns are the coefficients a_i , b_i , c_i , d_i of the cubic spline, S_i , joining the points x_i and x_{i+1} .
- Constraints: $S_i(x_i) = y_i$

$$\begin{aligned}a_1x_1^3 + b_1x_1^2 + c_1x_1 + d_1 &= y_1, \\a_2x_2^3 + b_2x_2^2 + c_2x_2 + d_2 &= y_2, \\&\dots \\a_{n-1}x_{n-1}^3 + b_{n-1}x_{n-1}^2 + c_{n-1}x_{n-1} + d_{n-1} &= y_{n-1}.\end{aligned}$$

- Constraints: $S_i(x_{i+1}) = y_{i+1}$

$$\begin{aligned}a_1x_2^3 + b_1x_2^2 + c_1x_2 + d_1 &= y_2, \\a_2x_3^3 + b_2x_3^2 + c_2x_3 + d_2 &= y_3, \\&\dots \\a_{n-1}x_n^3 + b_{n-1}x_n^2 + c_{n-1}x_n + d_{n-1} &= y_n.\end{aligned}$$



Finding Coefficients (cont.)

- Constraints: $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$

$$\begin{aligned}3a_1x_2^2 + 2b_1x_2 + c_1 - 3a_2x_2^2 - 2b_2x_2 - c_2 &= 0, \\3a_2x_3^2 + 2b_2x_3 + c_2 - 3a_3x_3^2 - 2b_3x_3 - c_3 &= 0, \\&\dots \\3a_{n-2}x_{n-1}^2 + 2b_{n-2}x_{n-1} + c_{n-2} - 3a_{n-1}x_{n-1}^2 - 2b_{n-1}x_{n-1} - c_{n-1} &= 0.\end{aligned}$$

- Constraints: $S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$

$$\begin{aligned}6a_1x_2 + 2b_1 - 6a_2x_2 - 2b_2 &= 0, \\6a_2x_3 + 2b_2 - 6a_3x_3 - 2b_3 &= 0, \\&\dots \\6a_{n-2}x_{n-1} + 2b_{n-2} - 6a_{n-1}x_{n-1} - 2b_{n-1} &= 0.\end{aligned}$$

- End point constraints: $S_i''(x_1) = 0$ and $S_{n-1}''(x_n) = 0$.

$$\begin{aligned}6a_1x_1 + 2b_1 &= 0, \\6a_{n-1}x_n + 2b_{n-1} &= 0.\end{aligned}$$



Example in Python

TRY IT! Find the cubic spline interpolation at $x = 1.5$ based on the data $x = [0, 1, 2]$, $y = [1, 3, 2]$.

First, we create the appropriate system of equations and find the coefficients of the cubic splines by solving the system in matrix form.

The matrix form of the system of equations is

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 8 & 4 & 2 & 1 \\ 3 & 2 & 1 & 0 & -3 & -2 & -1 & 0 \\ 6 & 2 & 0 & 0 & -6 & -2 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 12 & 2 & 0 & 0 \end{bmatrix} \begin{bmatrix} a_1 \\ b_1 \\ c_1 \\ d_1 \\ a_2 \\ b_2 \\ c_2 \\ d_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ 3 \\ 2 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

```
In [4]: b = np.array([1, 3, 3, 2, 0, 0, 0, 0])
        b = b[:, np.newaxis]
        A = np.array([[0, 0, 0, 1, 0, 0, 0, 0],
                       [0, 0, 0, 0, 1, 1, 1, 1],
                       [1, 1, 1, 1, 0, 0, 0, 0],
                       [0, 0, 0, 0, 8, 4, 2, 1],
                       [3, 2, 1, 0, -3, -2, -1, 0],
                       [6, 2, 0, 0, -6, -2, 0, 0],
                       [0, 2, 0, 0, 0, 0, 0, 0],
                       [0, 0, 0, 0, 12, 2, 0, 0]])
```



Example in Python (cont.)

```
In [5]: np.dot(np.linalg.inv(A), b)
```

```
Out[5]: array([[ -0.75],  
               [  0.  ],  
               [  2.75],  
               [  1.  ],  
               [  0.75],  
               [-4.5  ],  
               [  7.25],  
               [-0.5  ]])
```

- The two cubic polynomials
 - $S_1(x) = -0.75x^3 + 2.75x + 1$ for $0 \leq x \leq 1$ and
 - $S_2(x) = 0.75x^3 - 4.5x^2 + 7.25x - 0.5$ for $1 \leq x \leq 2$
- For $x=1.5$, we evaluate $S_2(1.5)$ whose estimated value is 2.7813.



Lagrange Polynomial Interpolation

- Finds a single polynomial that goes through all the data points, which is referred to as a Lagrange polynomial, $L(x)$.
- When computing Lagrange polynomials, it is useful to write them as a linear combination of Lagrange basis polynomials.
 - Lagrange basis polynomial

$$P_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

$$\begin{aligned} P_i(x_j) &= 1 \text{ when } i = j \\ P_i(x_j) &= 0 \text{ when } i \neq j \end{aligned}$$

- Lagrange polynomial

$$L(x) = \sum_{i=1}^n y_i P_i(x)$$

$$L(x_i) = y_i \text{ for every point as described above}$$



Lagrange Polynomial Interpolation in Python

TRY IT! Find the Lagrange basis polynomials for the dataset $x = [0, 1, 2]$ and $y = [1, 3, 2]$. Plot each polynomial and verify the property that $P_i(x_j) = 1$ when $i = j$ and $P_i(x_j) = 0$ when $i \neq j$.

$$P_1(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} = \frac{(x - 1)(x - 2)}{(0 - 1)(0 - 2)} = \frac{1}{2}(x^2 - 3x + 2),$$

$$P_2(x) = \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} = \frac{(x - 0)(x - 2)}{(1 - 0)(1 - 2)} = -x^2 + 2x,$$

$$P_3(x) = \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} = \frac{(x - 0)(x - 1)}{(2 - 0)(2 - 1)} = \frac{1}{2}(x^2 - x).$$



Lagrange Polynomial Interpolation in Python (cont.)

```
In [1]: import numpy as np
import numpy.polynomial.polynomial as poly
import matplotlib.pyplot as plt

plt.style.use("seaborn-poster")

In [2]: x = [0, 1, 2]
y = [1, 3, 2]
P1_coeff = [1, -1.5, .5]
P2_coeff = [0, 2, -1]
P3_coeff = [0, -.5, .5]

# get the polynomial function
P1 = poly.Polynomial(P1_coeff)
P2 = poly.Polynomial(P2_coeff)
P3 = poly.Polynomial(P3_coeff)

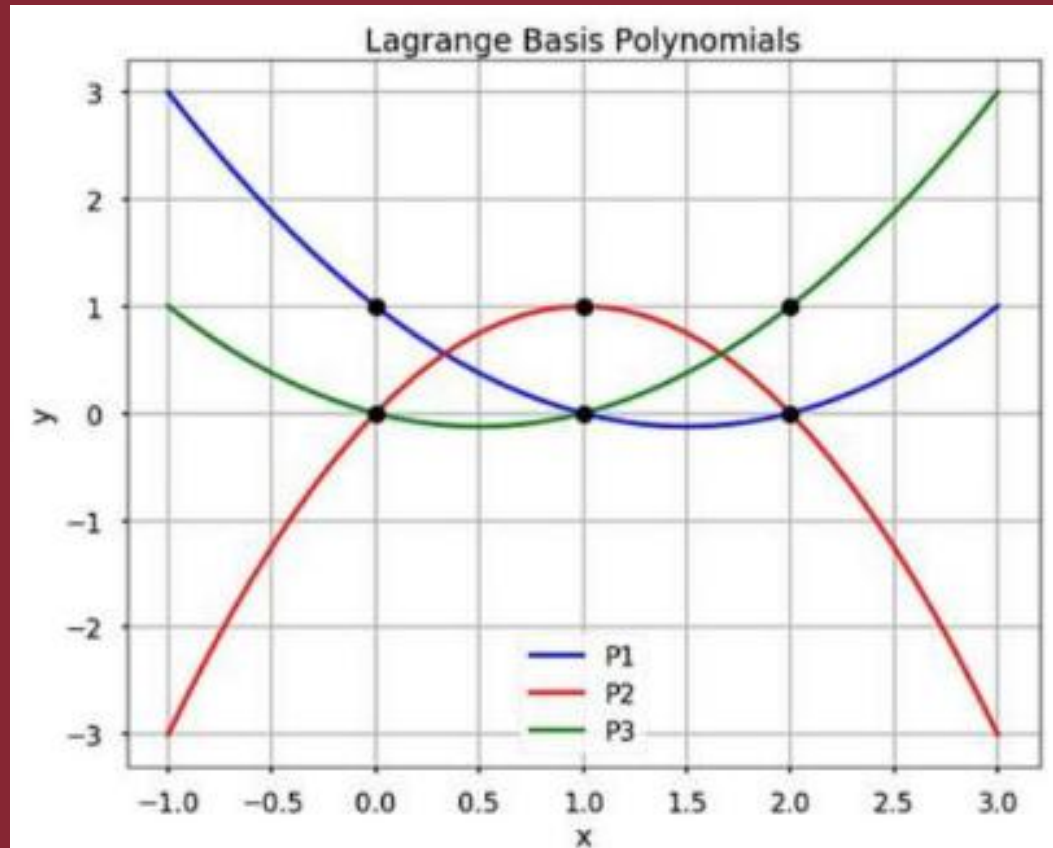
x_new = np.arange(-1.0, 3.1, 0.1)

fig = plt.figure(figsize = (10,8))
plt.plot(x_new, P1(x_new), "b", label = "P1")
plt.plot(x_new, P2(x_new), "r", label = "P2")
plt.plot(x_new, P3(x_new), "g", label = "P3")

plt.plot(x, np.ones(len(x)), "ko", x, np.zeros(len(x)), "ko")
plt.title("Lagrange Basis Polynomials")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



Lagrange Polynomial Interpolation in Python (cont.)



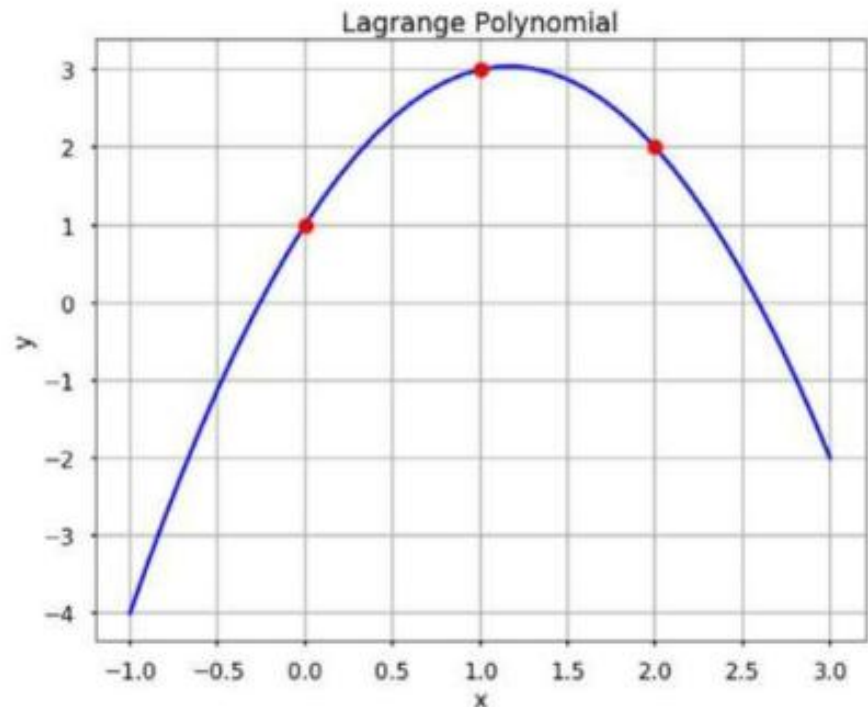


Lagrange Polynomial Interpolation in Python (cont.)

TRY IT! Using the previous example, compute and plot the Lagrange polynomial. Verify that it goes through each of the data points.

```
In [3]: L = P1 + 3*P2 + 2*P3
```

```
fig = plt.figure(figsize = (10,8))
plt.plot(x_new, L(x_new), "b", x, y, "ro")
plt.title("Lagrange Polynomial")
plt.grid()
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



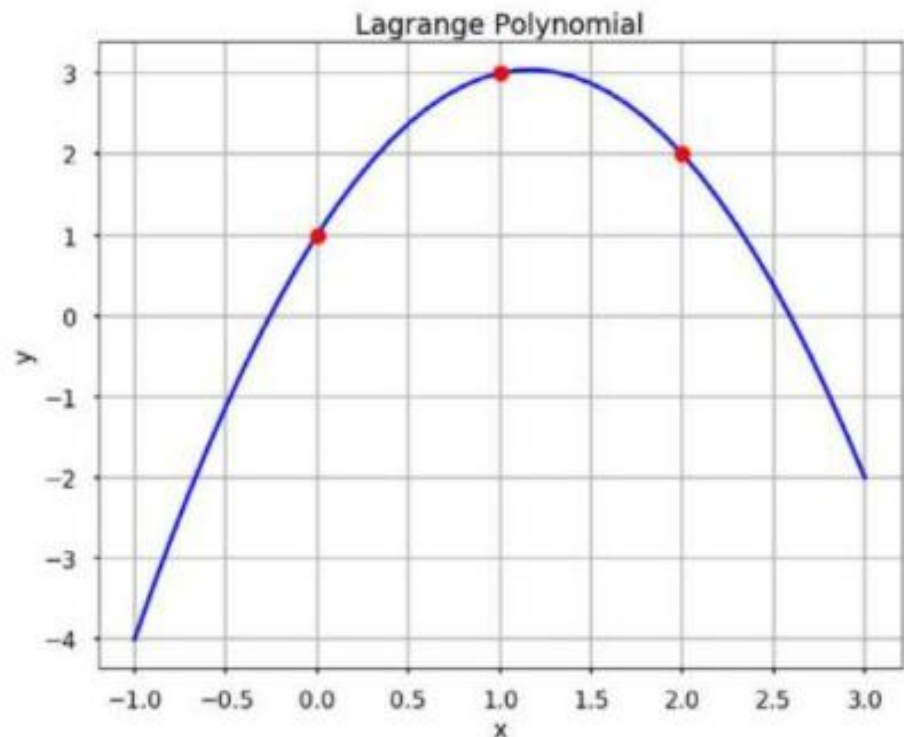


Lagrange Polynomial Interpolation in Python using Lagrange Function from Scipy

```
In [4]: from scipy.interpolate import lagrange
```

```
In [5]: f = lagrange(x, y)
```

```
In [6]: fig = plt.figure(figsize = (10,8))  
        plt.plot(x_new, f(x_new), "b", x, y, "ro")  
        plt.title("Lagrange Polynomial")  
        plt.grid()  
        plt.xlabel("x")  
        plt.ylabel("y")  
        plt.show()
```





Newton's Polynomial Interpolation

- Another popular way to exactly fit a set of data points.
- General form of the (n-1)th order Newton's polynomial that goes through n points

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_n).$$

- which can be re-written as

$$f(x) = \sum_{i=0}^n a_i n_i(x)$$

where

$$n_i(x) = \prod_{j=0}^{i-1} (x - x_j)$$

- Since the polynomial goes through each data point (x_i, y_i) , we have $f(x_i) = y_i \rightarrow f(x_0) = a_0 = y_0$ and $f(x_1) = a_0 + a_1(x_1 - x_0) = y_1$.
- We rearrange the equation to obtain $a_1 \rightarrow$

$$a_1 = \frac{y_1 - y_0}{x_1 - x_0}$$



Newton's Polynomial Interpolation (cont.)

- Keep going to obtain all the coefficients.

$$a_1 = \frac{y_1 - y_0}{x_1 - x_0}$$

$$a_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0}$$

$$a_3 = \frac{\frac{\frac{y_3 - y_2}{x_3 - x_2} - \frac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_1} - \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0}}{x_3 - x_0}$$

- See the pattern? These are called divided differences. If we define

$$f[x_1, x_0] = \frac{y_1 - y_0}{x_1 - x_0}, \text{ then}$$

$$f[x_2, x_1, x_0] = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} = \frac{f[x_2, x_1] - f[x_1, x_0]}{x_2 - x_1}.$$

- If we continue, we obtain the following iteration equations:

$$f[x_k, x_{k-1}, \dots, x_1, x_0] = \frac{f[x_k, x_{k-1}, \dots, x_2, x_1] - f[x_{k-1}, x_{k-2}, \dots, x_1, x_0]}{x_k - x_0}$$



Newton's Polynomial Interpolation (cont.)

- Advantage

- Once the coefficients are determined, adding new data points will not change the previously calculated coefficient.
- The following presents a divided difference table for finding the higher differences.

x_0	y_0				
		$f[x_1, x_0]$			
x_1	y_1		$f[x_2, x_1, x_0]$		
		$f[x_2, x_1]$		$f[x_3, x_2, x_1, x_0]$	
x_2	y_2		$f[x_3, x_2, x_1]$		$f[x_4, x_3, x_2, x_1, x_0]$
		$f[x_3, x_2]$		$f[x_4, x_3, x_2, x_1]$	
x_3	y_3		$f[x_4, x_3, x_2]$		
		$f[x_4, x_3]$			
x_4	y_4				



Newton's Polynomial Interpolation (cont.)

- From the divided difference table in the previous slide, we can calculate all coefficients and store them in a diagonal matrix.

y_0	$f[x_1, x_0]$	$f[x_2, x_1, x_0]$	$f[x_3, x_2, x_1, x_0]$	$f[x_4, x_3, x_2, x_1, x_0]$
y_1	$f[x_2, x_1]$	$f[x_3, x_2, x_1]$	$f[x_4, x_3, x_2, x_1]$	0
y_2	$f[x_3, x_2]$	$f[x_4, x_3, x_2]$	0	0
y_3	$f[x_4, x_3]$	0	0	0
y_4	0	0	0	0

- The first row in the matrix is all the coefficients that we need.



Newton's Polynomial Interpolation in Python

TRY IT! Calculate the divided difference table for $x = [-5, -1, 0, 2], y = [-2, 6, 1, 3]$.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

plt.style.use("seaborn-poster")

%matplotlib inline

In [2]: def divided_diff(x, y):
    """
    function to calculate the divided
    difference table
    """
    n = len(y)
    coef = np.zeros([n, n])
    # the first column is y
    coef[:,0] = y

    for j in range(1,n):
        for i in range(n-j):
            coef[i][j] = (coef[i+1][j-1]-coef[i][j-1])/(x[i+j]-x[i])

    return coef

def newton_poly(coef, x_data, x):
    """
    evaluate the Newton polynomial
    at x
    """
    n = len(x_data) - 1
    p = coef[n]
```

```
    for k in range(1,n+1):
        p = coef[n-k] + (x -x_data[n-k])*p
    return p
```

```
In [3]: x = np.array([-5, -1, 0, 2])
y = np.array([-2, 6, 1, 3])
# get the divided difference coef
a_s = divided_diff(x, y)[0, :]
```

```
# evaluate on new data points
x_new = np.arange(-5, 2.1, .1)
y_new = newton_poly(a_s, x, x_new)
```

```
plt.figure(figsize = (12, 8))
plt.plot(x, y, "bo")
plt.plot(x_new, y_new)
```

Out[3]: [

